

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 02 -

ARQUITETURA DE MICROSERVIÇOS E PRODUTIVIDADE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	28
REFERÊNCIAS.....	29

LEANDRO

O QUE VEM POR AÍ?

Imagine o contraste entre uma cidade. De um lado, o modelo monolítico: todos os serviços essenciais (correios, banco, hospital) concentrados em um único e imenso mega prédio. Se uma parte precisa de manutenção, o edifício inteiro é paralisado. De outro, a abordagem de microserviços: cada serviço em seu próprio prédio separado, autônomo, mas interconectado em uma rede eficiente.

Esta aula tem como foco explorar a transição fundamental dessas arquiteturas. Você entenderá por que e quando os microserviços se tornaram a solução para desafios de escala e produtividade. Analisaremos casos de sucesso, como Amazon e Netflix, que migraram de sistemas monolíticos para malhas de serviços extremamente ágeis.

Um ponto crucial será como o desacoplamento de um modelo de Machine Learning pode otimizar a velocidade das entregas de software.

HANDS ON

Nessa disciplina avançaremos da teoria para a prática, mostrando como projetar e consumir APIs de forma robusta.

Ao final desta aula, você estará apto(a) a tomar decisões estratégicas sobre o uso de microserviços para alavancar a produtividade da sua equipe, especialmente no contexto de implantação de modelos de IA em produção.

EMANIP

SAIBA MAIS

Aplicações Monolíticas vs. Microserviços: Fundamentos e Diferenças

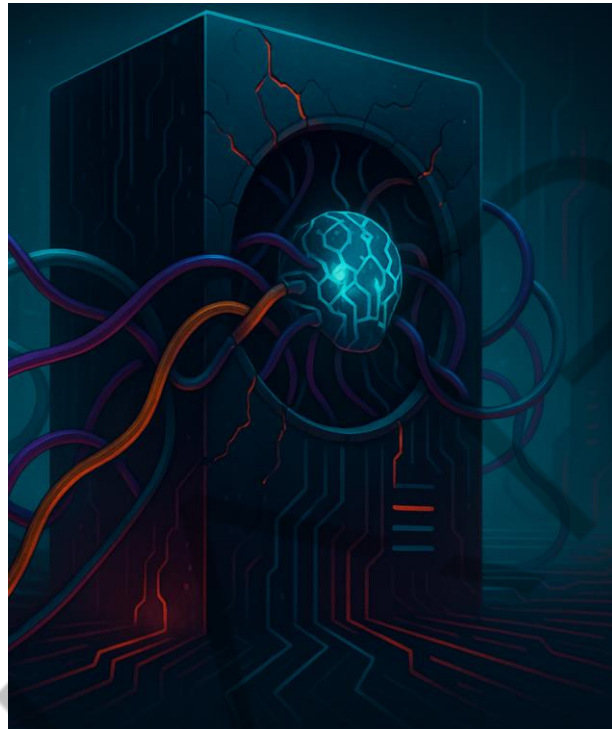


Figura 1 – Monolite
Fonte: Elaborado pelo autor (2026)

Uma aplicação monolítica é construída como um único bloco coeso: nela, todos os componentes (interface, lógica de negócio, acesso a dados etc.) vivem na mesma base de código e são implantados juntos como uma unidade indivisível. Se qualquer parte do sistema é modificada, toda a aplicação precisa ser reimplantada.

Em contraste, a arquitetura de microsserviços propõe dividir a aplicação em vários serviços pequenos, autônomos e focados em capacidades de negócio específicas, que podem ser desenvolvidos e implantados de forma independente. Cada microsserviço roda em seu próprio processo e comunica-se com os demais por interfaces bem definidas (chamadas HTTP/REST, mensagens em fila etc.), ao contrário do monólito, onde chamadas de função direta e compartilhamento de memória são comuns.

Para ilustrar, pense em um sistema de e-commerce. Em um design monolítico, módulos como catálogo de produtos, carrinho de compras, pagamentos e recomendações ficam todos dentro de um único aplicativo. Qualquer mudança no módulo de pagamentos exigiria fazer deploy de todo o sistema novamente. Já em uma arquitetura de microserviços, poderíamos ter serviços separados: um dedicado a catálogo, outro ao carrinho, outro para pagamentos, outro para recomendações e assim por diante. Cada serviço tem sua própria base de código e pode ser atualizado ou escalado de forma independente, sem impactar diretamente os outros. Essa divisão segue o princípio da responsabilidade única em escala arquitetural: cada microserviço cuida de um contexto de negócio bem definido e nada mais.

É importante notar que monolítico não é sinônimo de “ruim” ou “ultrapassado”. Monólitos têm vantagens de simplicidade, por exemplo: facilitam o reuso de código interno (tudo está no mesmo processo, então chamar funções ou compartilhar classes é trivial, sem a complexidade de chamadas remotas). Muitos desenvolvedores e desenvolvedoras experientes alertam que um monólito bem construído pode ser a opção mais sensata em muitos cenários.

O autor Sam Newman, referência em microserviços, observa que arquiteturas monolíticas continuam sendo uma escolha válida e muitas vezes são o padrão sensato: ele mesmo diz que precisa “ser convencido do porquê usar microserviços, em vez de procurar razões para não usá-los”. Ou seja, microserviços não são bala de prata universal, trazem benefícios claros, mas também adicionam complexidade. A decisão deve sempre considerar o contexto do projeto, o tamanho da equipe e os objetivos do negócio.

Resumindo: a diferença fundamental entre as abordagens está no grau de acoplamento e na forma de implantação. No monólito, todos os componentes vivem e morrem juntos, isto é, formam um único artefato de deploy e um único runtime. Na arquitetura de microserviços, cada componente é um serviço independentemente implantável, colaborando com os outros através de interfaces estáveis.

Essa separação permite uma série de vantagens em escalabilidade, robustez e ritmo de desenvolvimento, mas também impõe desafios significativos de

coordenação e infraestrutura. Nas próximas seções, vamos dissecar tanto os ganhos quanto os custos dessa mudança de paradigma.

Vantagens dos Microserviços

Escalabilidade granular

Em vez de escalar a aplicação inteira como um monólito, podemos escalar somente os serviços que demandam mais recursos. Por exemplo, se o serviço de busca de produtos for o gargalo de desempenho, aumentamos apenas a capacidade (número de instâncias, CPU, memória) desse microserviço, sem tocar nos demais. Em sistemas monolíticos, muitas vezes “é preciso escalar tudo junto”, mesmo que só uma parte precise de mais desempenho; com serviços menores podemos “escalar apenas aqueles que precisam, executando outras partes em hardware menos potente”.

Essa escalabilidade sob medida torna o uso de recursos mais eficiente, podendo reduzir custos operacionais.

Independência de deploy

Cada microserviço pode ser implantado ou atualizado isoladamente, sem interromper toda a aplicação ou fazer um “reempacotamento” geral. Isso aumenta significativamente a agilidade de desenvolvimento; assim, diferentes times podem liberar novas funcionalidades em seus respectivos serviços a qualquer momento, de forma assíncrona.

Boas práticas sugerem: cada microserviço deve ter seu próprio build e pipeline de CI/CD; evite situações em que seja necessário construir todos os serviços juntos, pois isso prejudica a independência de deploy. Empresas como Amazon e Netflix adotaram essa filosofia para reduzir impedimentos na hora de colocar software em produção, acelerando o ritmo de entregas.

Isolamento de falhas (robustez)

Uma arquitetura de microsserviços naturalmente compartimenta o sistema, servindo como “anteparas” contra falhas, referência às anteparas de navios que isolam compartimentos para o navio não afundar inteiro.

Se um serviço específico cair ou apresentar problemas, isso não precisa derrubar todo o sistema; apenas aquela funcionalidade fica degradada, enquanto o restante continua operando. Em um monólito, uma exceção não tratada ou sobrecarga em um módulo crítico pode indisponibilizar a aplicação inteira. Nos microsserviços, a falha de um componente tende a ser isolada e tratável de forma graciosa (por exemplo, exibindo uma mensagem de erro apenas naquela funcionalidade ou usando um valor padrão temporário), melhorando a tolerância a falhas global.

Essa robustez adicional, porém, depende de projetar o sistema para lidar inteligentemente com falhas. É famosa a frase: “um sistema distribuído é aquele em que a falha de um computador que você nem sabia que existia pode tornar o seu computador inutilizável”. Em outras palavras: problemas de comunicação podem causar efeitos em cascata inesperados se não houver mecanismos de contenção. Por isso, práticas como circuit breakers (disjuntores), timeouts e retentativas exponenciais são frequentemente adotadas para garantir resiliência em microsserviços.

Liberdade tecnológica

Cada microsserviço, por ser autônomo, pode ser implementado com a tecnologia que melhor se adequa ao seu propósito, sem ficar preso a um único stack do monólito. Por exemplo, podemos ter um serviço escrito em Python para algoritmos de Machine Learning convivendo com outro serviço em Java para processamento transacional pesado, desde que se comuniquem via APIs padronizadas. Em um monólito qualquer experimento tecnológico impacta todo o sistema, enquanto com múltiplos serviços “há vários lugares novos para tentar uma tecnologia”. Posso escolher um microsserviço de menor risco e usá-lo ali, limitando qualquer impacto negativo potencial.

Essa liberdade, quando bem administrada, promove inovação e uso de ferramentas especializadas dentro da empresa. Grandes plataformas como Netflix e Twitter concentram boa parte de seus serviços em uma stack comum, mas não restringem totalmente as escolhas, permitindo múltiplas linguagens e frameworks quando necessário para tirar proveito de soluções específicas

Desafios e Complexidade dos Microserviços

Se por um lado os microserviços trazem ganhos, por outro eles não vêm de graça. Operar com dezenas (ou centenas) de serviços distribuídos adiciona uma complexidade considerável em comparação a gerir um único sistema monolítico. É como sair de um computador central para um formigueiro de computadores menores cooperando e coordenar esse formigueiro dá trabalho. Orquestrar os serviços, garantir que se comuniquem corretamente, observar o comportamento emergente do conjunto e manter as equipes sincronizadas requer esforço consciente e investimento em infraestrutura.

Na prática, adotar microserviços exige uma maturidade maior em DevOps. Sem uma forte automação, a produtividade pode até cair inicialmente: configurações, deploys, monitoramento, tudo fica distribuído e multiplicado. Somente com investimento pesado em pipelines robustos de CI/CD, testes automatizados abrangentes, monitoração centralizada e infraestrutura elástica é que a promessa de rapidez e flexibilidade dos microserviços se materializa. Em suma, estamos trocando complexidade de desenvolvimento (no monólito) por complexidade de operação (nos microserviços).

Como destaca Sam Newman, microserviços demandam “compreensão das tecnologias de suporte em um grau muito maior”, se ignorarmos detalhes de implantação, redes ou monitoramento isso pode minar todo o projeto. Antes de adotar essa arquitetura, é preciso avaliar se sua organização está preparada para gerenciar a complexidade adicional e se os ganhos esperados realmente justificam os investimentos necessários.

Vale lembrar também que nem toda aplicação precisa de microserviços. Para projetos simples ou equipes muito pequenas, um monólito modular pode ser mais pragmático no começo. O Nubank, por exemplo, decidiu começar já com microserviços desde o dia 1, contrariando a recomendação clássica de “comece com monólito e depois refatore”. Eles descobriram que isso os tornou mais lentos no início, pois montar uma infraestrutura complexa de provisionamento enquanto criavam o produto do zero foi pesado para a pequena equipe, e só após algum tempo o trabalho fluiu melhor.

Por outro lado, como o negócio não exigia mudanças constantes no início, investir em uma base sólida cedo compensou quando a empresa escalou nos anos seguintes. Ou seja, há um trade-off: microserviços impõem um custo inicial maior em troca de benefícios de longo prazo em escala.

Avalie os sintomas no seu contexto: deploys demorados, dificuldade de escalar partes específicas, equipes esbarrando umas nas outras em entregas? Esses são sinais de alerta de que talvez seja hora de extrair componentes para serviços separados. Se não há tais dores, uma arquitetura mais simples pode atender melhor e mais rápido.

Em resumo, a adoção de microserviços deve vir acompanhada de uma mudança cultural e de ferramentas. Conceitos de “you build it, you run it” (você constrói, você opera), popularizados pela Amazon, tornam-se a norma: as mesmas equipes desenvolvedoras devem se responsabilizar por colocar o serviço em produção e mantê-lo funcionando. Isso incentiva a criação de softwares mais confiáveis e fáceis de manter, mas exige capacitação em operações e monitoramento.

Equipes de infraestrutura precisam prover plataformas internas para facilitar a vida de devs (ex.: sistemas de deploy self-service, observabilidade unificada). Foi o que fez a Amazon ao criar internamente o AWS, e o Nubank ao obsessivamente automatizar tudo na sua nuvem desde cedo. A recompensa, quando se acerta, é desbloquear a escala e a autonomia dos times, mas é preciso estar ciente do percurso e dos custos até lá.

Casos Reais de Adoção: Netflix e Amazon

Dei alguns spoilers, mas nada melhor para entender o impacto dos microsserviços do que examinar casos reais de empresas que trilharam esse caminho. Amazon e Netflix são frequentemente citadas como pioneiras na transição de monólitos legados para arquiteturas altamente distribuídas. Vamos ver como cada uma abordou esse desafio e quais ganhos obtiveram em produtividade, escalabilidade e inovação.

Amazon: Two-Pizza Teams e autonomia para inovar



Figura 2 – AWS-Two-Pizza Teams-Jeff
Fonte: Sen (2023)

No início dos anos 2000, a Amazon operava principalmente com uma arquitetura monolítica para seu site de comércio eletrônico. Conforme a empresa cresceu, esse monólito tornou-se um gargalo tanto para a escalabilidade quanto, sobretudo, para a velocidade de desenvolvimento. Equipes diferentes esbarravam umas nas outras ao tentar entregar novas funcionalidades, pois tudo estava interligado e dependente.

Empower innovation by limiting the size of teams

The concept of Amazon's two-pizza teams is straightforward: no team should be big enough that it would take more than two pizzas to feed them. Ideally, this is a team of less than 10 people: smaller teams minimize lines of communication and decrease overhead of bureaucracy and decision-making. This allows two-pizza teams to spend more time focusing on their customers, and constantly experimenting and innovating on their behalf — the biggest priority of high-performing teams at Amazon.

Smaller teams also increase ownership and empowerment. This can mitigate the Ringelmann Effect: a tendency for individual productivity to decrease in larger groups. As team resources swell, there is less focus on individual effort as people lean more on others to shoulder the load. The magnitude of individual contribution decreases as team size grows. Inversely, individual effort increases as the team size decreases.

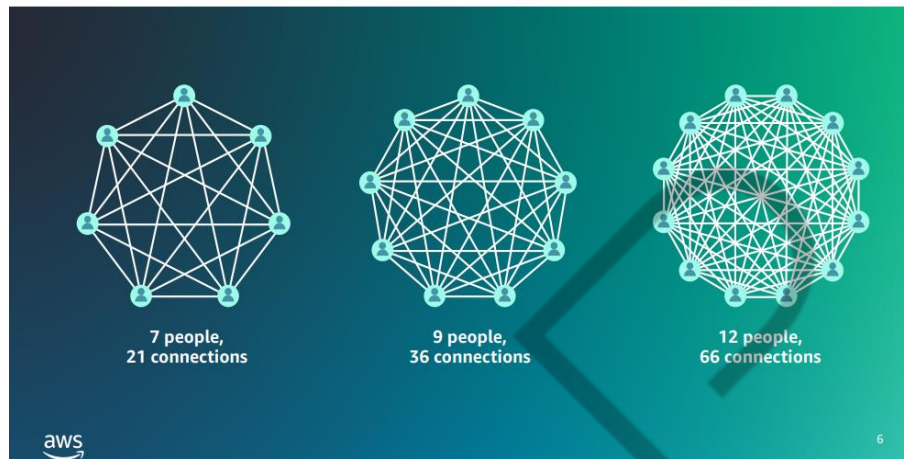


Figura 3 – AWS-Two-Pizza Teams
Fonte: Amazon Web Services (2023)

Em resposta, a Amazon, sob liderança de Jeff Bezos, instituiu um famoso mandato interno em 2002: toda comunicação entre times deve ocorrer via serviços (APIs), não mais por acessos diretos a banco de dados, chamadas internas de métodos ou compartilhamento de memória. Esse mandato (hoje lendário como “Bezos API Mandate”) foi o germe da transformação da Amazon rumo a uma arquitetura orientada a serviços (precursora dos microsserviços modernos). Em suma, cada equipe teria que expor seus dados e funcionalidades somente através de interfaces bem definidas e qualquer outra forma de integração seria proibida.

Organizacionalmente, a Amazon adotou o conceito das equipes de “duas pizzas”. A regra era que nenhum time de desenvolvimento deveria ser tão grande que duas pizzas não pudessem alimentá-lo, o que seriam na prática times de no máximo 8–10 pessoas. Mais importante que o número em si foi a ideia de que pequenos times são mais ágeis e devem ter autonomia total sobre o ciclo de vida do seu serviço. Ou seja, cada equipe desenvolve, testa, implanta e opera sua parte do sistema, de ponta a ponta. Esse modelo “você constrói, você opera” exigiu que a Amazon investisse

fortemente em ferramentas internas e foi daí que surgiu a necessidade de plataformas como o Amazon Web Services (AWS).

Inicialmente, o AWS nasceu para uso interno, oferecendo infraestrutura e componentes comuns para permitir que cada equipe fosse autossuficiente (por exemplo, armazenamento S3, banco de dados DynamoDB, filas SQS etc.). Com o tempo, claro, o AWS virou um produto comercial para clientes externos, mas vale lembrar que sua origem esteve ligada a capacitar os times da própria Amazon a trabalharem de forma descentralizada.

Os ganhos em agilidade foram notáveis. Cada pequeno time, dono de um serviço bem definido e conhecendo intimamente seu domínio de negócio, podia implementar mudanças e fazer deploy várias vezes ao dia, independentemente dos outros times. A coordenação pesada deu lugar a uma abordagem modular e paralela. Evidentemente, houve também efeitos colaterais: nos primeiros anos dessa migração, o site da Amazon apresentou alguma inconsistência visual e de experiência de usuário, já que diferentes módulos (mantidos por times distintos) implementavam telas e fluxos à sua maneira, sem um controle central rígido.

A Amazon considerou essa inconsistência de interface um preço justo a pagar pela velocidade e independência obtidas. Como colocou um dos líderes na época, era uma extensão lógica da busca da Amazon em reduzir a coordenação interna ao máximo, a ponto de propositalmente limitar interações entre times para não travar a inovação. Em outras palavras, preferiu-se um ecossistema heterogêneo e ágil a um monólito perfeitamente consistente, porém lento nas entregas.

Hoje, a Amazon é vitrine de uma arquitetura orientada a microsserviços. Cada página do site, digamos, a página do produto, é montada pela composição de múltiplos serviços de backend (recomendações, reviews, estoque, frete, pagamentos etc.), cada um escalando conforme a demanda em períodos de pico como a Black Friday.

A resiliência também aumentou: um erro no serviço de recomendações, por exemplo, não derruba todo o processo de checkout. Serviços menos críticos podem falhar isoladamente sem comprometer o núcleo do site. Em suma, a experiência da Amazon mostrou que quebrar um monólito em serviços menores, aliado a uma

reorganização da empresa em pequenos times autônomos, pode liberar enorme produtividade e inovação.

A contrapartida foi construir todo um aparato tecnológico para suportar isso e aceitar alguns trade-offs de padronização. É famoso o mantra atribuído à Amazon: “You build it, you run it” – ou seja, quem desenvolve o serviço é responsável por mantê-lo rodando em produção, o que incentiva práticas de código de qualidade, automação de testes e monitoramento proativo. Com essa cultura e infraestrutura, a Amazon conseguiu escalar sua engenharia de software junto com o negócio, lançando novos produtos e recursos em alta velocidade.

Não por acaso, a Amazon ajudou a moldar o pensamento moderno sobre APIs e microsserviços ao demonstrar, na prática, que tais princípios permitem escalar tanto a tecnologia quanto as equipes de desenvolvimento.

Netflix: Escala global e resiliência com Chaos Engineering

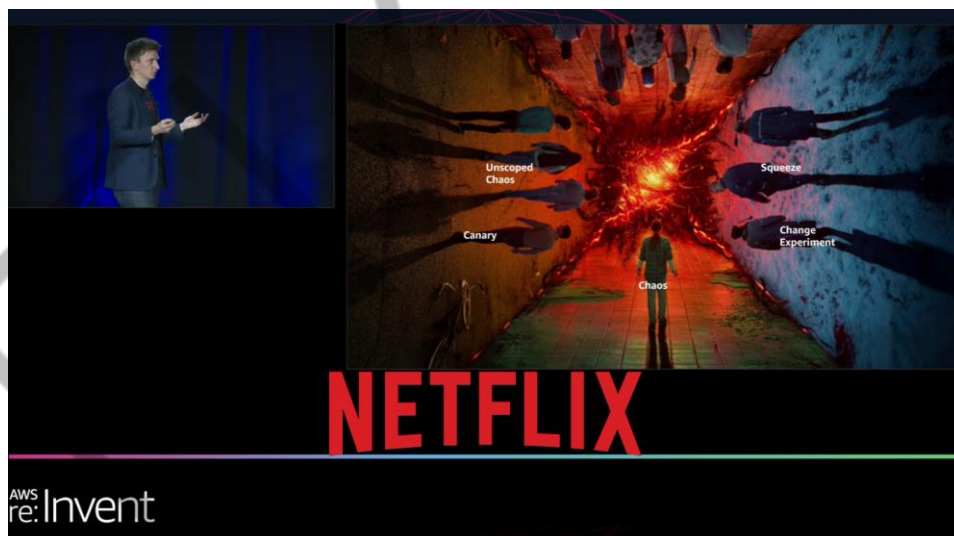


Figura 4 – Netflix: Escala global e resiliência com Chaos Engineering
Fonte: AWS Events (2022)

A Netflix, líder mundial em streaming, seguiu um caminho parecido em busca de escalabilidade e resiliência. Originalmente um serviço de entrega de DVDs pelo correio, a Netflix transformou-se em uma empresa de streaming digital em meados

dos anos 2000. Nesse processo, migrou de datacenters próprios para a nuvem AWS e adotou massivamente microsserviços para suportar seu crescimento explosivo e necessidade de disponibilidade 24/7 globalmente.

Um catalisador dessa mudança foi um grande outage (apagão) que a Netflix sofreu em 2008, quando um problema no banco de dados relacional central (parte do sistema monolítico da época) interrompeu o serviço por horas, algo catastrófico para uma empresa de mídia online. Após esse evento, a Netflix decidiu re-arquitetar tudo para a nuvem, decompondo funcionalidades em serviços independentes e distribuindo-os em múltiplas regiões e provedores. O objetivo declarado era nunca mais ter um ponto único de falha capaz de derrubar todo o serviço.

A cultura de engenharia da Netflix abraçou os microsserviços juntamente com uma filosofia de “projetar para falhas”. Eles popularizaram práticas de teste de resiliência em produção a ponto de desenvolverem a famosa suíte Simian Army, cujo membro mais célebre é o Chaos Monkey.

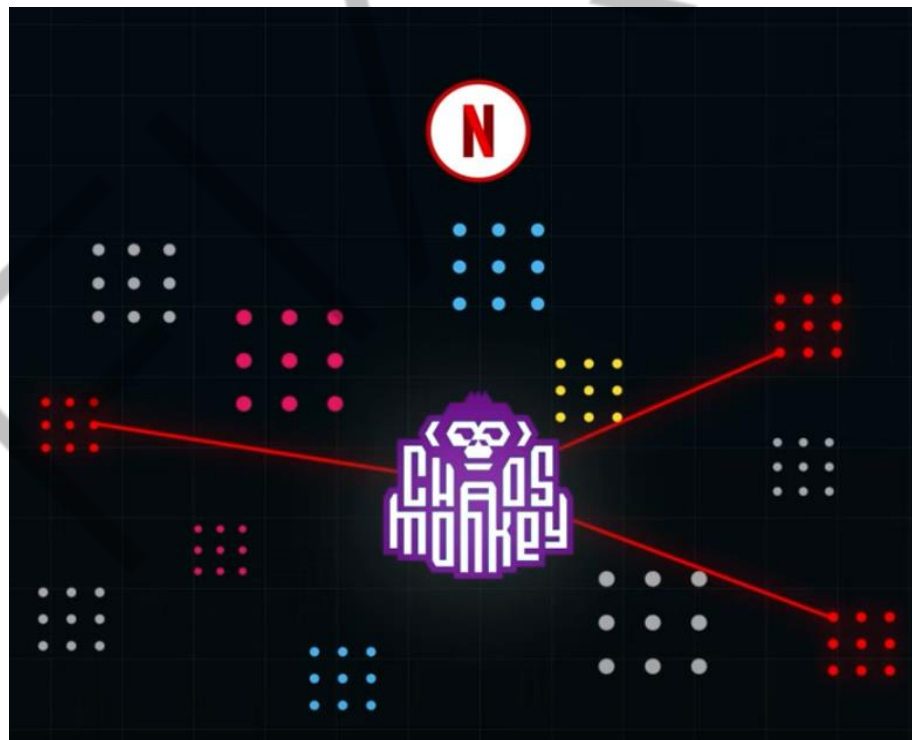


Figura 5 – Netflix: Chaos Monkey
Fonte: AWS Events (2022)

O Chaos Monkey é uma ferramenta que, de propósito, derruba aleatoriamente instâncias de serviço em produção (geralmente durante horário comercial, quando engenheiros estão de prontidão). Pode parecer contraintuitivo “causar falhas” deliberadamente, mas a ideia brilhante por trás disso é testar constantemente se o sistema é realmente resiliente como foi projetado para ser. Se o desligamento aleatório de uma máquina causa indisponibilidade para o usuário final, é sinal de que aquele componente não estava adequadamente redundante ou tolerante a falhas, ou seja, um bug arquitetural a ser corrigido. Basicamente, a Netflix decidiu não confiar na sorte: se você quer ter certeza de que aguenta falhas, cause falhas e veja o que quebra.

Com essa abordagem arrojada, a Netflix passou a incitar falhas para garantir que seus sistemas tolerem esses eventos. A iniciativa funcionou: ferramentas como Chaos Monkey (que “mata” serviços aleatoriamente) e Chaos Gorilla (que simula a queda de um datacenter inteiro) tornaram-se instrumentos do dia a dia. Isso acostumou os engenheiros e engenheiras a projetarem cada serviço já imaginando sua possível falha e recuperação. “A Netflix usa um serviço chamado Chaos Monkey para simular falhas... Basicamente, o Chaos Monkey é um serviço que mata outros serviços”, assim resumiu um engenheiro, explicando que a empresa queria que todos ficassem confortáveis com um nível constante de falhas acontecendo. Os serviços deveriam se recompor automaticamente, sem intervenção manual, para realmente atingir alta disponibilidade.

Graças a esses testes extremos, a Netflix atingiu um nível de confiabilidade impressionante: eles podem afirmar que aguentam perder servidores (ou até datacenters inteiros) sem interromper o streaming para os clientes. A arquitetura de microserviços, com muitos serviços redundantes rodando em vários lugares, foi o habilitador disso. Seria praticamente impossível conseguir o mesmo resultado com um banco de dados central único sustentando tudo.

Além da resiliência, a Netflix também ganhou agilidade organizacional semelhante à Amazon. Elas adotaram princípios de pequenos times autônomos inspirados no modelo das “two-pizza teams”. Cada funcionalidade do produto (por exemplo, o serviço de recomendação de filmes, ou o serviço de perfis de usuário)

passou a ser mantida por um time dedicado, que pode iterar rapidamente e escolher as tecnologias que achar melhores para aquela missão.

A Netflix tradicionalmente usou muito Java no backend, mas algumas APIs foram escritas em Node.js e atualmente misturam até Go, Python e outras linguagens quando faz sentido, contanto que cada serviço se integre nos protocolos padrão da plataforma (geralmente HTTP/JSON e gRPC). Ou seja, assim como a Amazon, eles deram liberdade tecnológica aos times para inovar, desde que respeitassem contratos de comunicação.

Internamente, a Netflix investiu em um rico ecossistema de bibliotecas cliente e ferramentas para suportar seus microserviços. Por exemplo, criou a biblioteca [Hystrix](#) (link github, hoje open source) para implementar circuit breakers e fallbacks de forma padronizada nas chamadas entre serviços. Criou também soluções de descoberta de serviços (Eureka), gateway inteligente (Zuul), sistema de deploy contínuo (Spinnaker), entre outras utilidades – muitas das quais foram liberadas à comunidade depois. Essas bibliotecas padronizaram como os serviços se comunicavam e como lidavam com falhas, facilitando a escalabilidade e a estabilidade do sistema como um todo.

Com o tempo, contudo, a Netflix percebeu que depender demais de bibliotecas compartilhadas também criava certo acoplamento entre os serviços. A solução evoluiu para arquiteturas ainda mais descentralizadas, como o uso de service mesh (malha de serviços), que desloca funcionalidades de comunicação para um nível infraestrutural mais transparente, reduzindo a necessidade de cada serviço importar bibliotecas pesadas.

Em síntese, a Netflix mostrou ao mundo como microserviços, aliados a uma cultura radical de teste de falhas, podem suportar uma operação global com zero interrupção mesmo diante de grandes problemas. Eles foram um dos primeiros casos de migração bem-sucedida de monólito para microserviços em nuvem e inspiraram muitas outras empresas a seguir o exemplo. Hoje, empresas brasileiras como iFood, Mercado Livre e Nubank também empregam arquiteturas de microserviços para escalar seus produtos digitalmente, cada uma adaptando as lições de pioneiros como Netflix e Amazon à sua realidade local.

Microserviços na integração de modelos de Machine Learning

Além de habilitar escalabilidade para negócios tradicionais, a arquitetura de microserviços pode impulsionar a produtividade em cenários de Inteligência Artificial e Machine Learning (ML).

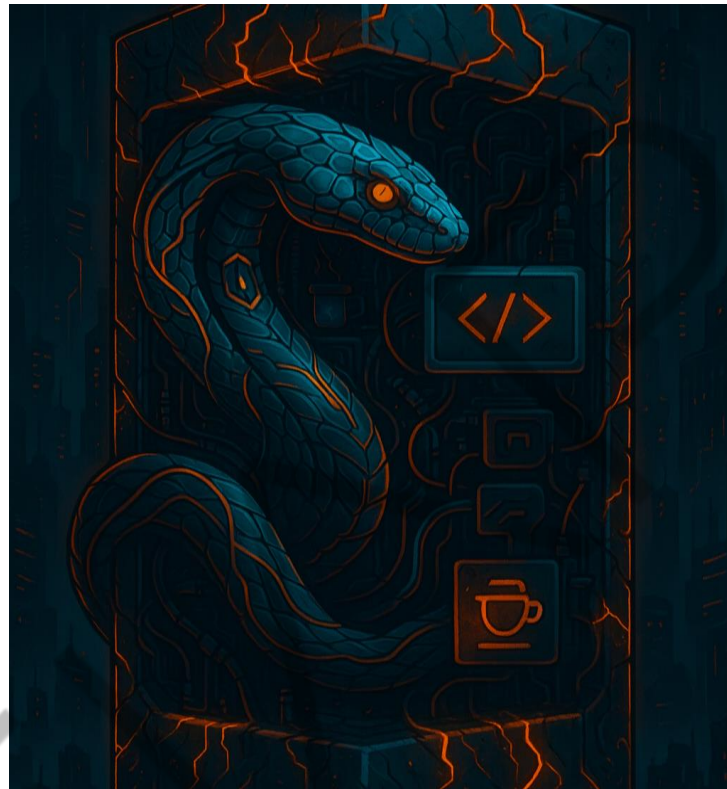


Figura 6 – Integração?
Fonte: AWS Events (2022)

Imagine que sua empresa desenvolveu um modelo de ML poderoso como, por exemplo, um modelo de recomendação de produtos ou de detecção de fraudes. Como integrar esse modelo à aplicação principal de forma escalável e sustentável? A abordagem tradicional seria embutir o modelo diretamente no código do sistema (um monólito) ou acoplá-lo fortemente a ele. Porém, isso frequentemente gera obstáculos: ciclos de deploy diferentes, linguagens distintas (muitas vezes o modelo é treinado em Python enquanto o sistema web está em outra linguagem), consumo de recursos específico do modelo (CPU/GPU) que não afeta o resto da aplicação do mesmo jeito etc.

Os microserviços oferecem um caminho elegante para colocar modelos em produção, desacoplando o ciclo de vida do modelo do software principal. Em vez de enterrar o código de inferência de ML dentro de um monólito, podemos criar um microserviço de inferência dedicado, com sua API própria (REST, gRPC etc.), que recebe dados e devolve previsões do modelo.

Esse serviço de ML roda isoladamente, possivelmente até em um servidor ou container separado, otimizado para cargas de ML, enquanto o restante do sistema (frontend, backend transacional) continua em sua tecnologia preferida. Essa separação traz diversas vantagens.

Desacoplamento de ciclos de vida

O modelo de ML pode evoluir e ser implantado independentemente do aplicativo principal. A equipe de dados/MLOps consegue fazer melhorias ou substituir o modelo (v1, v2, v3...) sem precisar sincronizar deploys com a equipe de produto. Novas versões do modelo entram em produção rapidamente, após validação, enquanto o restante do sistema permanece intocado. Em outras palavras, modelagem e desenvolvimento de produto andam em ritmos próprios, o que maximiza a produtividade de ambos os times.

Escalação e otimização de recursos

Serviços de inferência de ML costumam ter requisitos de hardware bem específicos como, por exemplo, uso intenso de CPU ou até necessidade de GPUs, além de muita memória para carregar modelos grandes. Ao isolá-los em microserviços, podemos escalar verticalmente (dando máquinas mais potentes) ou horizontalmente (mais instâncias) apenas esses serviços de ML, em vez de ter que escalar toda a aplicação monolítica por causa de um componente.

Alocamos máquinas especiais (instâncias com GPU, por exemplo) somente onde necessário e deixamos o resto do sistema em servidores comuns. Assim, cada serviço pode escalar independentemente, o que é uma forma eficiente de usar

recursos da cloud, atingindo um balanceamento adequado entre demanda e capacidade sem gastos desnecessários.

Interoperabilidade e reuso

Um modelo exposto via API REST ou gRPC pode facilmente servir a múltiplos consumidores diferentes como o site web, o aplicativo móvel, sistemas internos de análise, parceiros externos etc.

Em vez de duplicar código de inferência em cada uma dessas aplicações, tudo converge naquele serviço único e aplicações escritas em linguagens distintas podem consumir o modelo da mesma forma, fazendo requisições à API padronizada. Essa abordagem de “modelo como serviço” também facilita a composição de modelos em pipelines: por exemplo, um serviço pode chamar outro internamente (um sistema de recomendação pode consumir resultados de um serviço de perfis de usuário e assim por diante), construindo soluções mais complexas de forma modular.

Liberdade de stack e atualização tecnológica

Assim como outros microserviços, um serviço de ML pode usar a ferramenta e linguagem ideais para seu propósito. Em geral, isso significa poder aproveitar o riquíssimo ecossistema Python para IA: bibliotecas de última geração, frameworks especializados (TensorFlow, PyTorch, scikit-learn etc.), sem impactar nada fora daquele serviço. Se amanhã surge uma biblioteca nova promissora, a equipe de IA consegue testá-la naquele microserviço, sem esbarrar no conservadorismo que um monólito impõe.

Isso evita o “lock-in” tecnológico de ter todo o sistema preso a uma única linguagem ou ambiente. Ao deployar modelos de ML como microserviços com endpoints de API, cientistas de dados podem usar “qualquer framework – TensorFlow, PyTorch ou Keras – sem se preocupar com compatibilidade de stack”, eliminando amarras tecnológicas e permitindo evolução rápida do ambiente de IA.

Manutenção e foco

Microserviços de ML tendem a ter escopos bem definidos; por exemplo, um serviço classificador de imagens, outro detector de fraude, outro recomendador de filmes. Cada qual é relativamente pequeno em código e responsabilidade, o que os torna mais fáceis de entender e manter isoladamente do que se todo o código de ML estivesse misturado dentro de um grande aplicativo. Para novos membros na equipe, é muito mais simples um onboarding focando em um serviço de previsão específico do que ter que compreender um monólito inteiro para mexer no modelo embutido nele. Essa divisão por competências de negócio melhora a clareza arquitetural e distribuição das tarefas de manutenção.

Experimentação controlada e versionamento

Ao separar o modelo em seu próprio serviço, torna-se factível implementar estratégias de teste A/B ou lançamentos canários para novas versões do algoritmo.

Por exemplo, você pode implantar a versão 2 do modelo de recomendação lado a lado com a v1 no microserviços e configurar o Gateway para direcionar 10% do tráfego de previsões para a v2, mantendo 90% na v1. Assim, avalia-se se a nova versão melhora as métricas de negócio antes de se promover completamente. Em um monólito, esse nível de controle fino é muito mais difícil de orquestrar. Microserviços também facilitam o rollback: se um novo modelo apresentar problema, basta reverter o deploy daquele serviço específico para a versão anterior, algo que pode ser feito em minutos, sem afetar o restante do sistema. Essa independência reduz drasticamente o risco de atualizações de modelos “quebrarem” a aplicação inteira.

Em resumo, separar modelos de ML como microserviços tende a habilitar tanto ganhos de velocidade (ao liberar o fluxo de trabalho de cientistas de dados) quanto de qualidade (ao possibilitar testes mais controlados e arquitetura otimizada).

Não é à toa que frameworks modernos de MLOps assumem esse padrão: muitos já pressupõem que haverá um endpoint dedicado para o modelo e provêm ferramentas para implantar modelos em containers com APIs prontas.

Dito isso, sempre é bom reforçar: microsserviços não são obrigatórios. Há cenários simples em que servir um modelo dentro de um monólito pode ser suficiente, especialmente se o modelo for leve e o volume de chamadas baixo.

No contexto de ML, se você tem um MVP para colocar no ar rapidamente, integrar o modelo em um aplicativo existente pode agilizar o começo. Entretanto, à medida que o sistema cresce em escala ou os modelos ficam mais complexos, a arquitetura de microsserviços torna-se praticamente indispensável para manter a produtividade e a confiabilidade. Felizmente, com ferramentas como FastAPI, Flask, Django etc., criar um microsserviço atualmente não é um bicho de sete cabeças: em pouco tempo você “empacota” uma funcionalidade em um web service robusto. E com plataformas em nuvem facilitando o deploy (seja via orquestradores como Kubernetes, seja via funções serverless), o overhead operacional diminui a cada dia.

Variação Serverless: Microsserviços com FastAPI e AWS Lambda

Uma variação moderna da arquitetura de microsserviços utiliza a computação serverless para implementar os serviços. Serverless (em português “sem servidor”, embora servidores existam nos bastidores) significa executar código sob demanda sem a necessidade de gerenciar servidores dedicados. O provedor de nuvem se encarrega de alocar recursos somente quando a função é invocada. Isso se harmoniza com a filosofia de microsserviços, levando a granularidade ao extremo, às vezes sendo chamado de “microsserviços funcionais” ou FaaS (Function as a Service).

Por exemplo, em vez de manter um serviço web de recomendações rodando 24/7 em contêineres, é possível empacotar a lógica de recomendação como uma função Lambda na AWS. Essa função é ativada apenas quando chega uma requisição, geralmente através de um API Gateway que mapeia uma rota HTTP para a função Lambda. O AWS Lambda instancia o código em questão de milissegundos, processa a requisição e retorna a resposta.

Se houver 1000 requisições concorrentes, o provedor automaticamente escala para múltiplas execuções paralelas da função, podendo distribuí-las em vários

servidores distintos, sem que o(a) desenvolvedor(a) precise orquestrar nada. Esta é a escalabilidade automática por design: quando há mais carga, mais funções Lambda são disparadas em paralelo; quando não há tráfego, nenhuma função roda, resultando em custo zero enquanto ocioso.

No contexto de aplicações Python e APIs REST, surgiu uma biblioteca chamada Mangum que permite rodar aplicações ASGI, incluindo FastAPI, dentro do AWS Lambda. Basicamente, ela traduz o evento recebido do API Gateway, que contém a requisição HTTP, em um objeto ASGI padrão, passa esse objeto para o aplicativo FastAPI e captura a resposta para devolver via API Gateway. Com isso, é possível escrever uma API com FastAPI de forma normal, definindo endpoints, modelos Pydantic e lógica de negócio e implantá-la como uma função Lambda serverless.

O API Gateway age como a "porta de entrada", roteando caminhos HTTP para a função, e o Lambda executa sob demanda. Essa combinação une a produtividade do FastAPI, um framework web em Python rápido e extremamente amigável, com os benefícios operacionais do serverless: não se preocupar em gerenciar servidores ou contêineres, pagar somente pelo tempo de execução (o que é econômico para workloads com tráfego irregular) e ter escalabilidade praticamente infinita pronta para uso.

O FastAPI tem se tornado popular em cenários serverless exatamente por sua eficiência e tempo de inicialização muito rápido. Essas são características importantes quando o código pode ser carregado sob demanda. Muitas empresas e desenvolvedores(as) usam FastAPI com AWS Lambda para servir modelos de ML em produção de forma serverless, conseguindo escala automática sem precisar manter clusters Kubernetes continuamente. Imagine que seu modelo de recomendações recebe picos de acesso às 20h e quase nenhum acesso às 22h. Com um deploy serverless, às 20h o provedor escalará seu serviço de ML para quantas funções paralelas forem necessárias. Já às 22h, com baixo uso, ele reduz a escala para zero instâncias. Tudo isso automaticamente, cobrando apenas pelos minutos em que as funções rodaram. Essa elasticidade dinâmica é perfeita para muitos casos de uso de IA.

Outra variante relacionada é o container serverless, como AWS Fargate ou Azure Container Instances, que permite rodar microserviços containerizados sem gerenciar VMs ou nós. Você define o container da sua aplicação e diz à nuvem: "execute este container quando necessário, escale conforme a demanda" e o provedor cuida do resto. Isso é útil para casos em que uma função Lambda pura não basta como, por exemplo, quando é necessário maior controle sobre o ambiente de runtime ou tempos de execução mais longos do que os poucos minutos suportados por funções serverless.

Em termos de produtividade, o modelo serverless pode acelerar entregas porque elimina a necessidade de provisionar infraestrutura e lida automaticamente com grande parte da orquestração. Contudo, exige atenção especial na fase de teste e depuração. Como não há um servidor constante, simular o ambiente localmente é mais complexo e ferramentas específicas como o AWS SAM, Serverless Framework, etc. são úteis para rodar funções em modo de desenvolvimento.

Também é preciso estar ciente dos limites impostos pelos provedores, por exemplo: tempo máximo de execução por invocação, quantidade de memória disponível por função e latência de cold start (a primeira chamada que instancia a função pode ser mais lenta). Um bom conjunto de testes e monitoração mitigará esses desafios.

Resumindo, serverless é uma opção atraente para implementar microserviços em determinados cenários, pois oferece escala automática e operação simplificada. Com frameworks leves como FastAPI, criar uma API robusta e delegar à nuvem a tarefa de executá-la sob demanda ficou muito viável. Para serviços de inferência de ML, isso significa que você pode disponibilizar um modelo via FastAPI em um endpoint HTTP e deixar que o provedor escale de acordo com as consultas.

API Gateway e Load Balancer em arquiteturas de microserviços

Ao migrarmos de um monólito para um conjunto de microserviços, dois componentes de infraestrutura ganham destaque no desenho do sistema: o API Gateway e o Load Balancer (balanceador de carga). Eles têm papéis distintos e

complementares para garantir que as requisições dos clientes cheguem corretamente aos serviços apropriados no backend.

O API Gateway é uma camada intermediária que geralmente fica na "fronteira" do sistema, recepcionando todas as chamadas externas (vindas de aplicativos cliente, navegadores, chamadas de terceiros etc.) e encaminhando-as aos microserviços internos adequados. Pense nele como a porta de entrada unificada do seu conjunto de serviços.

Em vez de expor dezenas de URLs de serviços diferentes diretamente ao cliente (o que seria complexo e inseguro), expõe-se um único endpoint público, ou um pequeno conjunto deles, no Gateway. Ele faz o roteamento inteligente internamente. Por exemplo, suponha que o cliente faça uma requisição GET `/api/v1/produtos/123`. O API Gateway pode reconhecer o prefixo `/produtos` e encaminhar essa chamada para o microserviço de catálogo; se outra requisição vier para `/api/v1/pedidos/`, ele roteia para o serviço de pedidos, e assim por diante.

Além de roteamento, gateways normalmente implementam várias preocupações transversais (cross-cutting concerns), como autenticação, autorização, limite de requisições (rate limiting), cache de respostas, agregação de resultados de múltiplos serviços e até tradução de protocolos. Um caso comum é a agregação de respostas: o cliente móvel faz uma única requisição ao Gateway (`/api/v2/dashboard`, por exemplo) e o Gateway internamente chama múltiplos microserviços (perfil, notificações, recomendações etc.) para compor uma resposta unificada, aplicando o padrão Backends for Frontends (BFF).

Com isso, simplifica-se a vida do cliente (menos idas e vindas) e otimiza-se a comunicação. Em resumo, o API Gateway fornece uma fachada sob medida para os clientes e, ao mesmo tempo, protege e gerencia o tráfego que entra no ecossistema de microserviços.

Já o Load Balancer atua no contexto de um único serviço, mas com múltiplas instâncias dele. Em microserviços, é comum termos várias réplicas de cada serviço rodando (em contêineres ou VMs), tanto por motivos de escala para atender mais requisições simultâneas quanto de redundância, para tolerar falhas individuais. O balanceador de carga recebe as requisições direcionadas a um serviço específico e

as distribui entre as instâncias disponíveis, geralmente usando alguma estratégia (round-robin, menor número de conexões etc.) para equilibrar o uso.

Por exemplo, se temos 5 instâncias do microserviço de pagamentos rodando, o load balancer assegura que nenhuma fique sobrecarregada enquanto outras estão ociosas. Ele reparte as transações de forma equilibrada. Além disso, um bom balanceador monitora a saúde das instâncias: se uma delas cai ou fica inativa, ele a retira de rotação, direcionando o tráfego apenas às instâncias saudáveis remanescentes.

Nos ambientes modernos de orquestração, como Kubernetes, muito desse trabalho de balanceamento já é feito automaticamente pelo orquestrador, que tem componentes próprios para service discovery e distribuição de carga. Mas conceitualmente, o load balancer garante que a escalabilidade horizontal seja transparente para quem consome o serviço. Para o cliente ou para o Gateway, não importa se existem 2 ou 200 instâncias atrás de um endereço. Todas são acessadas por um único endpoint virtual e o balanceador cuida de distribuir as chamadas.

Um exemplo concreto: a arquitetura clássica de microserviços na Netflix/Amazon usava o Nginx ou ferramentas semelhantes na ponta como proxy reverso e load balancer. Internamente, para alguns serviços críticos, podia haver níveis adicionais de balanceamento. Hoje, ferramentas modernas como o Traefik combinam funções de proxy reverso, balanceador de carga e integração com orquestradores em um pacote único.

Do ponto de vista do cliente externo, o API Gateway muitas vezes atua como um balanceador de carga, roteando as chamadas para múltiplas instâncias dos microserviços. Em outras configurações, o Gateway roteia para o serviço e o serviço tem seu próprio Load Balancer dedicado (comum em nuvens públicas). Em arquiteturas serverless, o provedor da nuvem assume a função de gateway e balanceador.

Em resumo, Gateways e Load Balancers são cruciais: o Gateway oferece uma interface única e segura, aplicando políticas. O Load Balancer garante que cada serviço use sua capacidade total e permaneça disponível, mesmo com falhas de

instâncias. Juntos, eles escondem a complexidade interna e oferecem um sistema coeso ao consumidor.

Desvendamos aqui a essência da transição de arquiteturas monolíticas para a flexibilidade e escala dos microsserviços. O foco não foi apenas na tecnologia, mas na forma como a reorganização do software e das equipes, como nos casos da Amazon e Netflix, impulsiona a produtividade e a resiliência. Você está agora munido(a) dos conceitos fundamentais para entender como essa arquitetura se aplica estrategicamente, especialmente para desacoplar e escalar modelos de Machine Learning.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos a evolução da arquitetura de software de aplicações monolíticas para microserviços, sempre com foco em produtividade e qualidade, e passamos pelas diferenças fundamentais, principais vantagens, desafios, custos e cases reais.

Você aprendeu que microserviços são uma ferramenta poderosa para escalar não apenas a aplicação, mas também as dinâmicas de trabalho de equipes de desenvolvimento, permitindo que múltiplos times entreguem valor simultaneamente, cada um focado em sua parte.

Viu também que esse modelo traz novos desafios de coordenação e infraestrutura que precisam ser endereçados para colher os benefícios. No contexto de Machine Learning, discutimos que microserviços podem ser o alicerce de uma estratégia de MLOps eficaz, ao separar modelos em serviços flexíveis e escaláveis.

Ao concluir esta aula, você deve ser capaz de avaliar de forma crítica quando um monólito bem estruturado atende às necessidades e quando os microserviços começam a fazer diferença para a produtividade e escalabilidade do sistema. E caso opte por microserviços, você agora conhece seus principais pontos de atenção.

REFERÊNCIAS

AMAZON WEB SERVICES. **Powering Innovation and Speed with Amazon's Two-Pizza Teams**. 2023. Disponível em: https://d1.awsstatic.com/onedam/marketing-channels/website/aws/en_US/whitepapers/approved/executive-insights/two-pizza-teams-ebook.pdf. Acesso em: 14 jan. 2026.

AWS EVENTS. **AWS re:Invent 2022 – The evolution of chaos engineering at Netflix (NFX303)**. 2022. Disponível em: <https://www.youtube.com/watch?v=Xbn65E-BQhA>. Acesso em: 14 jan. 2026.

GÉRÓN, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. Beijing: O'Reilly Media, 2023.

GIFT, N.; DEZA, A. **Practical MLOps: Operationalizing Machine Learning Models**. Sebastopol: O'Reilly Media, 2021. LUBANOVIC, B. **FastAPI: Modern Python Web Development**. Sebastopol: O'Reilly Media, 2023.

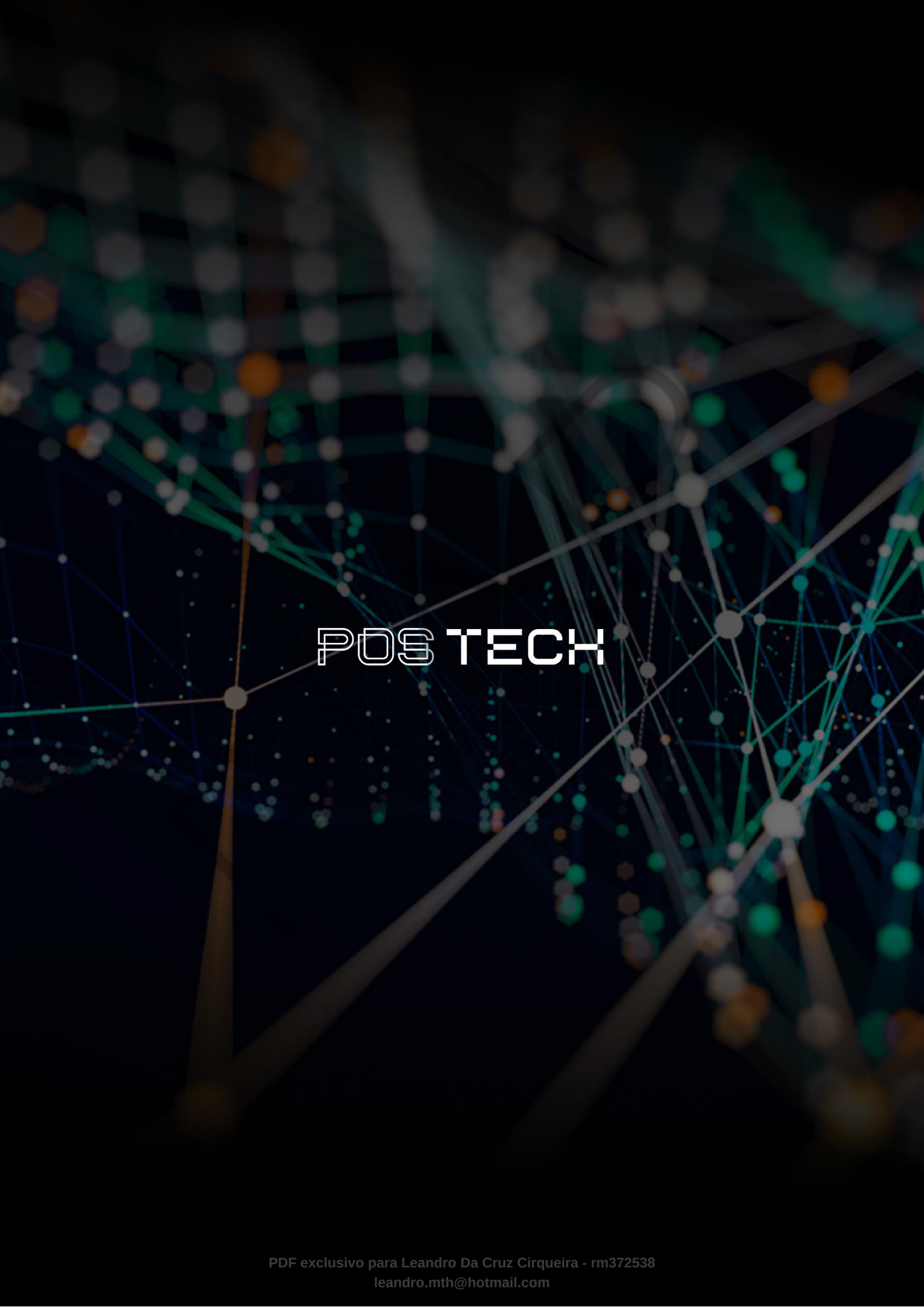
NEWMAN, S. **Building Microservices: Designing Fine-Grained Systems**. Sebastopol: O'Reilly Media, 2021.

SEN, S. **Jeff Bezos' Two-Pizza Rule: Maximise Team Productivity**. 2023. Disponível em: <https://yourstory.com/2023/11/jeff-bezos-two-pizza-rule-secret>. Acesso em: 14 jan. 2026.

PALAVRAS-CHAVE

Microserviços. Netflix. Amazon. Produtividade. Escalabilidade. API. WEB HTTP. REST. API RESTful. GraphQL. gRPC. JSON. MLOps.

EMANIP



POSTECH